

Day 3 — Production-Grade GenAI, RAG, Tokenisation, MCP and Observability

In-depth explanation of every concept, with analogies,
worked examples and delivery guidance

Written so that you can read it once and teach from it confidently across a full eight-hour day. Each concept has: what it means in plain terms, why it matters, the technical detail behind it, an analogy for the room, and the misunderstandings participants typically arrive with.

Enterprise AI Engineering Bootcamp · Companion to the 44-slide Day 3 presentation

How to Use This Document

Read end to end once before Day 3. Keep it beside you during delivery — sections map to slide numbers in order. Four kinds of box appear throughout: **Analogy**, **How to say it**, **Watch for** and **Deeper detail**.

The central argument of Day 3

When a RAG system gives a wrong answer, the overwhelming majority of the time the right passage was never retrieved. The model then did exactly what it should — answered from what it was given. Teams consistently respond by improving the prompt or changing the model, which addresses the last stage of a pipeline that failed at the second.

Everything today follows from that: ingestion quality caps retrieval quality, retrieval quality caps answer quality, and you cannot tell which stage failed unless you measure them separately.

Timing guide

Slot	Content	Pacing notes
09:00 – 10:00	Module 1 — tokenisation and context	Compressible if the room is strong. The lost-in-the-middle point is the one to protect
10:00 – 11:15	Module 2 — document ingestion	Show a badly-extracted table if you can. Nothing else makes the case as fast
11:15 – 12:00	Labs 1 and 2 combined	The top-20-passages result must be measured, not asserted
12:00 – 13:00	Module 3 — retrieval architecture	Hybrid and reranking are the two to spend time on
13:45 – 14:45	Lab 3 — strategy comparison	Every team leaves with a filled-in five-row table
14:45 – 15:45	Module 4 — MCP	Lead with the forty-versus-eight integration arithmetic
15:45 – 17:00	Module 5 — evaluation and observability	Do not squeeze this. The diagnosis table in 5.2 is the most actionable slide of the day
17:00 – 17:30	Module 6 — improvement patterns	Brisk. The ranked table does most of the work
17:30 – 18:30	Labs 5 and 6, assignment brief	Produces assignment deliverable 8 directly

Module 1 — Tokenisation and Context Engineering

1.1 What a token is, and why it matters commercially

A token is roughly three-quarters of a word in English, and considerably less in technical text full of part numbers and abbreviations.

Content	Approximate tokens	Note
One page of prose	500 – 700	The usual planning figure
One page of a technical manual with tables	900 – 1,400	Tables and part numbers tokenise poorly
A part number such as HX-4471-B	6 – 9	Alphanumerics fragment heavily
A tool schema definition	150 – 400 each	Charged on every request that offers the tool
An image	Hundreds to low thousands	Depends on resolution and model
A RAG request with 8 passages	6,000 – 12,000	Retrieved context dominates the bill

The two commercial consequences to state. "First, you are paying for retrieved context, not for the user's question — which means retrieval precision is a cost control as much as a quality control. Second, every tool schema you attach is charged on every single request whether the tool is used or not. Twenty tools attached to every call is a permanent tax."

Deeper detail — why part numbers tokenise badly. Tokenisers are trained on general text, so common English words map to single tokens while an alphanumeric string like HX-4471-B is split into fragments that carry no meaning individually. This has two effects: cost, and more importantly, retrieval quality — dense embeddings of fragmented identifiers are unreliable, which is a large part of why sparse keyword retrieval remains essential for technical content.

1.2 The context window is not free space

Teams assume a large context window means you can put everything in and let the model sort it out. What actually happens is that accuracy degrades as irrelevant context grows, latency rises roughly with input length, and cost rises directly.

The lost-in-the-middle effect. Across many models, information at the beginning or end of a long context is recalled substantially more reliably than information in the middle. Practical consequences: put the most relevant retrieved passage first, order chunks by relevance rather than document order, and keep the passage count small enough that ordering matters less. Passing twenty passages instead of five frequently reduces answer quality while tripling the cost.

Analogy — the briefing pack. Give an engineer a two-page brief containing exactly what they need and they will act on it correctly. Give them a two-hundred-page pack containing the same two pages somewhere in the middle and they will miss things — not because they are careless, but because the signal is diluted. The model behaves the same way, and for related reasons.

1.3 Chunking — the decision with the largest quality impact

Strategy	How it works	Good for	Weakness
Fixed size	N tokens with overlap	Uniform prose	Cuts tables and procedures mid-structure

Sentence-aware	Breaks on sentence boundaries	General documents	Ignores document structure
Paragraph	Breaks on paragraph boundaries	Well-structured text	Highly variable chunk sizes
Section-aware	Breaks on headings and hierarchy	Technical manuals	Requires reliable structure extraction
Layout-aware	Respects tables, figures, columns	Industrial documentation	Needs a capable parser
Parent-child	Small chunks to search, large to return	Precision plus completeness	More complex index and retrieval logic
Semantic	Breaks where meaning shifts	Mixed content	Slow to compute, harder to reason about

For service manuals and maintenance procedures, section-aware or parent-child almost always beats fixed size — and it is typically a one-day change.

1.4 Overlap, and the procedure that got cut in half

Overlap exists so that a passage split across a chunk boundary is still retrievable. Ten to fifteen per cent is the usual default.

The failure this prevents — describe it concretely. A safety procedure reads: "1. Isolate the supply. 2. Verify zero energy. 3. Apply the lock. 4. Attach the tag." A fixed 400-token chunk boundary falls between steps 2 and 3. A query about locking out the machine retrieves the second chunk, which begins at step 3. The answer omits isolation and verification entirely — and it is fluent, confident and dangerous. Section-aware chunking prevents this; overlap mitigates it.

How to use this example. It is the single best argument for structure-aware chunking, and it lands hard with an industrial audience because they can immediately see the consequence. Use it, and do not soften it.

1.5 Token budgeting

Component	Typical share	How to control it
System prompt and instructions	300 – 1,000	Written once, reviewed, versioned. Rarely the problem
Tool and MCP schemas	0 – 4,000	Attach only tools relevant to this request
Retrieved passages	2,000 – 10,000	The main lever. Rerank, then cut to three to five
Conversation history	0 – 5,000	Summarise older turns rather than carrying them verbatim
User question	20 – 200	Negligible
Reserved for the answer	500 – 2,000	Must be reserved explicitly or generation truncates

The pattern to repeat all day. "Retrieve twenty, rerank, pass four. Cheaper, faster and measurably more accurate — three improvements from one change. It is the most transferable thing you will learn today, and it appears again in Module 3 and Module 6."

Module 2 — Document Ingestion for Industrial RAG

2.1 Why industrial documents break naive extraction

Feature	What naive extraction does to it
Tables	Returns column text as a run-on sentence. The row-to-value relationship is destroyed, so torque values become dissociated from part numbers.
Multi-column layout	Extracts in the wrong reading order, interleaving two unrelated columns into one incoherent passage.
Figures and diagrams	Discarded entirely, along with the callout numbers that make the surrounding text meaningful.
Headers and footers	Repeated on every page. Left in, they pollute every chunk and dilute embeddings with boilerplate.
Numbered procedures	Flattened. Step numbering and hierarchy are lost, and steps become independent statements.
Scanned pages	Contribute nothing without OCR — and their absence is silent, with no error raised.

Delivery advice. Show one real badly-extracted table on the screen. Nothing else makes the case as quickly or as convincingly, and the room will remember it for the rest of the week.

2.2 Four levels of extraction

1. **Naive text.** Preserves words, roughly in order. Loses tables, columns, structure, images.
2. **Layout-aware.** Preserves reading order and paragraph structure. Still loses table semantics.
3. **Table-preserving.** Extracts tables as structured rows. Row-to-value relationships survive.
4. **Metadata-rich.** Adds section hierarchy, product family, version and access level. Enables filtering.

The finding to name in advance and let them verify. "Moving from level 1 to level 3 typically produces a larger accuracy improvement than any change to the model, the prompt or the embedding. It is unglamorous work and it is where the return is."

Watch for. Level 4 is not optional if Day 2's security model is to be implemented. Access-filtered retrieval requires access level as indexed metadata on every chunk. Adding it later means re-indexing the entire corpus — budget it as a migration.

2.3 The metadata that earns its place

Field	Enables	Consequence of omitting it
Source document and page	Citation and verification	Answers cannot be checked, so they will not be trusted
Section hierarchy	Parent-child retrieval	Chunks arrive without the heading that gives them meaning
Product family or model	Filtered retrieval	Procedures from the wrong variant returned confidently
Document version and status	Superseded-content suppression	The system cites a withdrawn procedure
Effective and expiry dates	Time-correct answers	Obsolete guidance presented as current
Access level	Role-filtered retrieval	Day 2's security model cannot be implemented

Language	Correct-language retrieval	Mixed-language results confuse retrieval and generation
Content type	Tuning retrieval by type	Tables and prose treated and ranked identically

2.4 Document versioning and supersession

In an industrial context this is a safety concern, not a housekeeping preference. Six practices: version every document as a distinct object; mark superseded content explicitly rather than deleting; filter to current by default with an override for historical queries; make status visible in the citation; trigger re-indexing on publication rather than nightly; and retain superseded content for audit and for equipment still running the older revision.

How to say it. "A retrieval system that confidently cites a withdrawn safety procedure is worse than one that finds nothing. If you take one thing from Module 2, take document status as an indexed, filterable field."

Module 3 — Retrieval Architecture

3.1 Dense, sparse and hybrid

Method	How it works	Strong on	Weak on
Dense (vector)	Semantic similarity of embeddings	Paraphrase, synonyms, concepts	Exact identifiers — may return a similar-looking part number
Sparse (BM25)	Term matching weighted by rarity	Part numbers, error codes, acronyms	Paraphrase — a differently-phrased question finds nothing
Hybrid	Both, with rank fusion	Both of the above	Two indexes and a fusion step to maintain

The argument that settles it in one sentence. "Your engineers type: 'what is the torque spec for the mounting bolts on HX-4471-B'. Dense retrieval handles the first half and can quietly fail on the second. Sparse handles the second and fails on the first. Nothing about that failure is visible — you get a plausible passage about a different part."

Deeper detail — rank fusion. The usual method is Reciprocal Rank Fusion, which combines two ranked lists using the reciprocal of each item's rank rather than its raw score. It is used because dense and sparse scores are not on comparable scales and normalising them reliably is difficult. RRF needs no tuning, which is a large part of why it is the default.

3.2 Reranking — the highest-return addition

Retrieve 30 to 50 candidates by hybrid search, rerank them with a cross-encoder, cut to the top three to five, then generate.

Deeper detail — why two stages. A bi-encoder embeds query and document separately, so it can never model their interaction — that limitation is exactly why it is fast enough to run over millions of documents. A cross-encoder processes the pair together and models the interaction directly, which is far more accurate and far too slow for a full corpus. Using each where it is strong gives you the recall of the first and the precision of the second, and it reduces cost because you pass four passages instead of twenty.

How to say it. "Reranking is usually the highest return per day of engineering work available to a RAG system. If you add one thing after this course, add this."

3.3 Filtering, rewriting and expansion

- **Metadata filtering.** Restrict candidates before search — product family, version, access level, date. Turns a hard ranking problem into an easy one by removing what should never have been a candidate. Frequently the largest single improvement and almost always underused, because it needs metadata captured at ingestion.
- **Query rewriting.** Resolve pronouns and conversational references, expand abbreviations, split compound questions. Measure it — it does not always help and can distort intent.
- **Query expansion.** Add synonyms and domain terms. Valuable where field vocabulary differs from documentation vocabulary. A domain glossary drives it.

3.4 Structural retrieval patterns

Pattern	How it works	Use when
---------	--------------	----------

Parent-child chunking	Search small chunks, return the larger parent	The best general default — precision in search, completeness in the answer
Hierarchical retrieval	Retrieve the section, then within it	Large structured documents
Multi-hop retrieval	One retrieval informs the next query	Questions needing facts from two documents joined
Graph RAG	Traverse entity relationships	Questions about how components or faults relate — powerful, frequently premature
Self-checking RAG	Verify the answer against passages before responding	High-stakes answers where unsupported claims are unacceptable
Retrieval fallback	A second strategy when the first scores weakly	Robustness
No-answer detection	Explicitly return that the corpus does not cover this	Always

Watch for — Graph RAG enthusiasm. It is genuinely powerful for relationship questions and it is expensive to build and maintain. Hybrid retrieval plus reranking plus metadata filtering usually gets further, faster, on industrial documentation. Push back gently and ask what question they cannot answer without it.

3.5 No-answer detection

Without it, retrieval always returns its top matches however weak, the model answers anyway, and the user gets a fluent, confident, wrong answer — with a citation to a document that does not support it, which makes it worse rather than better.

Implementation: a threshold on the reranker score, plus an explicit instruction that the model must answer only from the provided passages and must say so when they are insufficient.

Measurement: build a set of questions your corpus genuinely does not cover and measure how often the system correctly declines. That is no-answer accuracy.

The exercise that makes this land. "Write ten questions your documentation definitely does not answer — about equipment you do not make, or a procedure that does not exist. Run them. Most teams discover their system answers all ten confidently."

Module 4 — MCP for Tool and Data Integration

4.1 Why MCP matters for enterprise AI

The problem: every AI application integrates with the same enterprise systems and each writes its own bespoke integration. Five applications across eight systems is forty integrations, each with its own auth handling, error semantics, versioning and security review.

What MCP provides: a standard protocol between an AI application (client) and a capability provider (server). Eight servers, each built and reviewed once, consumed by any client.

Lead with this arithmetic. "Forty bespoke integrations of varying quality, or eight governed servers. MCP's real value in an enterprise is that it turns integration from an application concern into a platform concern. The team that owns the service-ticket system builds and governs one server; every AI application consumes it under the same contract."

4.2 Client and server roles

	MCP client	MCP server
Is	The AI application	A capability provider wrapping a system or data source
Owned by	The application team	The team that owns the underlying system
Responsible for	Deciding when to call, handling results, enforcing user identity	Exposing capabilities, validating inputs, enforcing its own access rules
Examples	A knowledge assistant; a diagnostic agent	Ticket lookup; document search; parts catalogue; telemetry query

Three primitives a server can expose: tools (actions the model may invoke), resources (data the client may read) and prompts (reusable templates).

Practical note on client support. Support for the three primitives varies by host application — several implement tools only. Design your servers so the tool interface alone is sufficient, and treat resources and prompts as enhancements. This keeps the server portable across whichever host application the organisation standardises on.

4.3 When to use MCP, and when not to

Use MCP when	Use a direct API call when
The model decides at runtime whether the capability is needed	The step always happens in a fixed pipeline
Several AI applications need the same capability	One application needs it, always the same way
The capability is owned by another team	You own both sides and the contract is stable
You want consistent tool-call logging and governance	The call is already inside a governed service

The distinction that stops overreach. "Does the model decide whether this happens? If not, it is not a tool. A retrieval step that always runs before generation is a pipeline stage — wrapping it in MCP adds latency and indirection for nothing."

4.4 Tool contracts

The model chooses tools by reading their descriptions. A badly described tool is not called, or is called wrongly, and neither failure is obvious.

Element	Requirement	Common failure
Name	Unambiguous and specific	Two tools whose names could each plausibly apply
Description	What it does, when to use it, when not to	Describes the implementation instead of the purpose
Input schema	Typed, each parameter described and constrained	Free-text parameters filled inconsistently
Output schema	Predictable structure	Raw system responses with inconsistent shape
Error semantics	Defined responses for not-found, unauthorised, unavailable	Exceptions leaving the model with nothing usable
Examples	One or two representative invocations	None — usage inferred from the name alone

The standard to give them. "Write tool descriptions for a competent new colleague who has never seen the system. That is precisely the reader you have."

4.5 and 4.6 Security for tool access

Six controls: propagate the caller's identity to the server; enforce authorisation at the server independently; allow-list tools per role outside the model; log every invocation; make state-changing calls idempotent; and gate irreversible actions on human approval.

Risk	How it manifests	Mitigation
Over-broad tool scope	A read-everything tool invoked for a restricted user	Least privilege; server-side authorisation per caller
Confused deputy	The server acts with its own high privilege on behalf of a low-privilege user	Propagate caller identity; never a shared service account
Injected tool invocation	Retrieved content induces a tool call	Tool authorisation derives from caller identity, never from content
Untrusted server	A server added without review receives sensitive arguments	Registry of approved servers; review before connection
Argument injection	Model-generated arguments used unsanitised downstream	The server validates and parameterises its own inputs
Silent data egress	A tool sends internal content externally	Explicit egress review per tool; outbound logging

Deeper detail — the confused deputy. A privileged component acting on behalf of an unprivileged caller without checking what that caller is allowed to do. It is the classic form of this class of vulnerability and it is very easily introduced in MCP architectures, because the natural implementation is for the server to hold one high-privilege credential to the underlying system. Identity must flow end to end and the server must make its own decision.

Module 5 — RAG Evaluation and Observability

5.1 The eight metrics

Metric	Question it answers	Diagnoses
Context recall	Did retrieval find the passages needed?	Retrieval — the usual culprit
Context precision	Of what was retrieved, how much was relevant?	Retrieval — cost and lost-in-the-middle
Faithfulness	Is the answer supported by the passages?	Generation — hallucination
Answer relevance	Does the answer address the question?	Generation — drift and evasion
Citation accuracy	Do citations actually support the claims?	Generation — the trust signal users check
Hallucination rate	How often are unsupported claims made?	Generation — what management asks about
No-answer accuracy	Does it decline correctly when coverage is absent?	System — almost never measured
Groundedness	Is every claim traceable to a source?	System — composite quality signal

5.2 The diagnosis table — the most useful thing in Module 5

Context recall	Faithfulness	Diagnosis	What to fix
Low	—	Retrieval failed. The answer never had a chance	Chunking, hybrid, reranking, metadata filtering
High	Low	The right passages were found and ignored	Prompt grounding, citation enforcement, verification
High	High, still wrong	The source document itself is wrong or outdated	Document versioning and supersession — a content problem
High	High, correct	The system is working	Measure cost and latency; examine failing slices

Why this matters. "This converts 'the RAG system gives bad answers' into a specific diagnosis in one step. The overwhelming majority of cases land in the first row — retrieval failed and the model was blamed. Teams routinely spend weeks on prompt engineering for a problem a reranker would have fixed in a day."

5.3 Building the RAG evaluation set

1. **Real questions first.** From support tickets, engineer queries and search logs. Invented questions are systematically easier than real ones.
2. **Record the correct passage, not just the answer.** This is what makes context recall measurable, and it is what separates a diagnostic evaluation set from a score you cannot act on.
3. **Include out-of-corpus questions.** Ten to twenty the documentation genuinely does not cover.
4. **Include per-role variants.** If retrieval is access-filtered, the correct answer differs by role.
5. **Include known-hard cases.** Ambiguity, multi-document questions, superseded content.
6. **Synthetic questions to fill gaps** — with an expert verifying every correct answer.

5.4 Traces and spans

A trace is the full record of one request; a span is one step within it. Eight spans worth recording: query received, query rewrite, retrieval, reranking, context assembly, tool calls, generation, post-processing.

The one to insist on. "The context-assembly span records exactly what was passed to the model, with token counts. When a user reports a wrong answer, that span usually answers the question immediately — and almost no proof of concept records it. It is cheap to add and it will resolve most of your support questions in one look."

5.5 The observability dashboard

Panel	Metrics	Alert on
Quality	Recall, precision, faithfulness, citation accuracy from scheduled runs	Any metric below threshold
No-answer behaviour	Decline rate and correctness of declines	Decline rate falling toward zero
Retrieval health	Score distribution, empty-result rate, filter hit rates	Rising empty results
Tool calls	Invocation rate, failure rate, latency per tool	Failure rate above threshold
Cost	Tokens and cost per request, per user, per tool	Sudden rise in tokens per request
Latency	p50, p95, p99 broken down by span	p95 breaching the objective
Feedback	Ratings, escalation rate, corrections	Rising negative feedback
Error clusters	Failures grouped by cause, ranked by frequency × cost	A new cluster appearing

Deeper detail — tokens per request as a quality signal. A sudden rise usually means retrieval quality dropped: scores fell, so the system pulled in more passages to compensate, or the fallback path started firing. It is one of the earliest warnings available and it is measurable without any labelled data, which makes it operationally valuable.

5.6 Regression testing

Triggered by any prompt, model, embedding, chunking or retrieval change, plus a scheduled weekly run regardless. The suite contains the gold set, every fixed production failure, out-of-corpus questions, per-role variants and known-hard cases. It only grows.

The compounding argument. "Every production failure that gets fixed becomes a permanent case. Within a year the suite encodes everything the system has ever got wrong, and no regression can silently reintroduce a past failure. That is the difference between a system that improves and one that oscillates."

Module 6 — RAG Accuracy Improvement Patterns

6.1 The thirteen levers, ranked

Lever	Typical return	Effort	Notes
Better chunking	High	Days	Section-aware over fixed. Often the largest single gain
Better metadata	High	Days-weeks	Enables filtering, often the largest gain after chunking
Hybrid retrieval	High	Days	Close to mandatory for technical content
Reranking	High	Days	Best return per day of work on this list
Citation enforcement	High	Days	Large trust gain; makes wrong answers checkable
Prompt grounding	Medium-high	Hours	Answer only from passages; say so when insufficient
Domain glossary	Medium-high	Days	Where field vocabulary differs from documentation
Failure-based tests	Medium-high	Ongoing	Prevents regression rather than improving the score
Query decomposition	Medium	Days	Only helps on genuinely compound questions
Answer verification	Medium	Days	Second pass checking claims. Doubles cost
Better embeddings	Medium	Weeks	Requires full re-index. Measure before committing
Human feedback loop	Medium, compounding	Weeks	Slow to start, largest long-run source of gain
Knowledge graph	Variable	Months	Powerful for relationships, frequently premature

How to present this. "Work top-down. Most teams reach for better embeddings or a knowledge graph while chunking is still fixed-size and there is no reranker. The ranking is deliberately ordered against what people instinctively try first."

6.2 Prompt grounding and citation enforcement

Prompt grounding: instruct explicitly to answer only from provided passages, say so when they are insufficient, and not use general knowledge. Cheap and effective — but it shifts the rate rather than eliminating the behaviour. One control among several.

Citation enforcement: every claim carries a reference, and the reference is then validated programmatically — that the cited passage was retrieved and does contain the claim. The validation step is what makes it real; unvalidated citations are frequently plausible and wrong, which is worse than none.

The adoption argument — use this with management. "An engineer who can check the source will use a system that is right 85% of the time. An engineer who cannot check will not use one that is right 95% of the time, because a wrong answer they cannot detect is worse than no answer. Citation accuracy is an adoption metric as much as a quality metric."

Labs 5 and 6 — Facilitation notes

Start from a deliberately weak pipeline. Apply improvements one at a time, measuring after each, and produce an ablation table exactly as on Day 1.

Facilitation. Some changes will not help on their data. Reporting that honestly is as valuable as reporting the wins — and it directly produces assignment deliverable 8, which carries into the Day 5 capstone.

Closing the Day

1. Most RAG failures are retrieval failures.
2. Ingestion quality caps everything downstream.
3. Retrieve broadly, rerank precisely, pass few.
4. Measure retrieval and generation separately.
5. Declining to answer is a feature, not a failure.

Close by asking each team which of the thirteen levers they will apply first. Most will name chunking or reranking, which is the right answer.

Companion documents: **Day 3 Presentation** (44 slides with speaker notes) and **Day 3 Question Bank** (anticipated participant questions with model answers).